

Balancing Robot Course Content

1. DC Motor Control: Theory, Motor Drivers, PWM signal generation
2. Incremental Encoders to estimate the position/velocity of the robot
3. Proportional-Integral-Derivative (PID Controller): theory and implementation
4. Moving average filter
5. RC Joystick Integration
6. Attitude estimation: tilt angle estimation using an IMU sensor
7. Linear Quadratic Regulator
8. Stepper Motor Integration
9. PCB Design

Mini-balancing robot Hardware

Small balancing robot with a PCB board:

1. PCB board
2. Pololu 50:1 micro metal gearmotor
3. Magnetic Encoder
4. Wheels

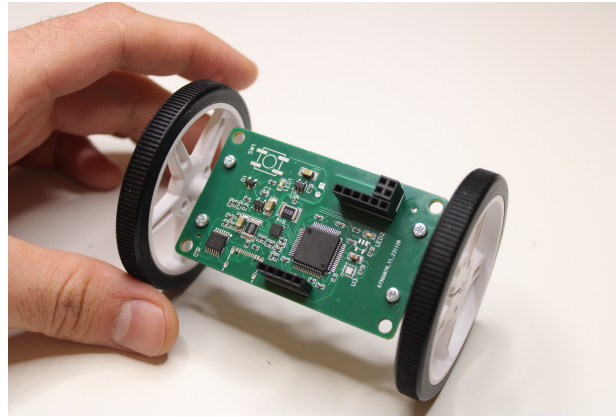


Figure: Mini-balancing robot

Custom Hardware

1. Balancing Robot kit
2. ICM-20948 IMU sensor Board
3. Nucleo-L476 Board
4. X-NUCLEO-IHM15A1 Motor Driver Board

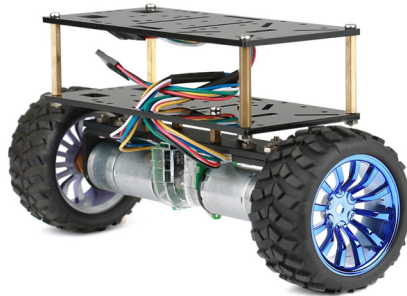


Figure: Balancing Robot Development Kit

CHAPTER 1: Motor Control

Outline of Chapter 1:

1. Motor Fundamentals.
2. STSPIN240 Motor Driver
3. Pulse-Width-Modulation (PWM) using the STM32 MCU to control the speed of a motor.
4. STM32 Timer encoder mode to read the velocity of a motor.
5. Moving Average Filter.
6. Proportional-Integral-Differential (PID) Motor Control.

Motor Types

1. DC brushed motor - Robotic small-scale vehicles
2. Brushless DC motor - Drones
3. Stepper motor - CNC Machine, 3d Printers



Figure: DC motor, Brushless DC Motor, Stepper Motor

Brushed DC motor Internal Structure

1. Magnet - stator
2. Coils with a shaft - rotor
3. Commutator and Brushes - ensure electrical connection

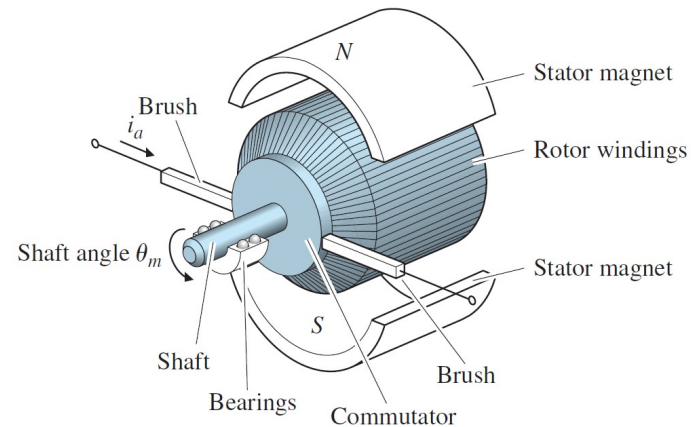
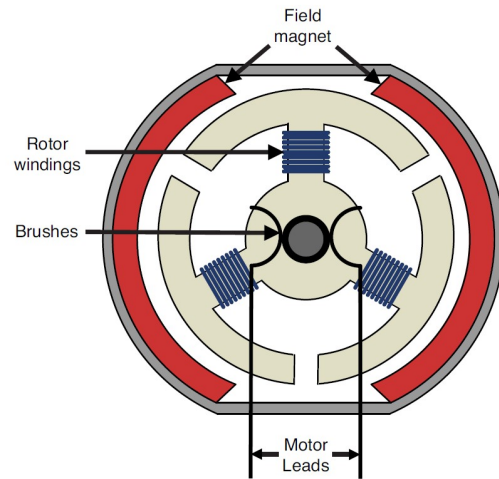


Figure: Internal Structure of the DC motor

Basic Principles: Torque and Rotational speed

Rotational Speed, angular velocity, defines how fast a motor is rotating. Its unit is rotation per minute (rpm, in practical cases) or radians/sec(SI unit).

Torque is a measure of the rotational force applied to an object, causing it to rotate around an axis. In the context of a motor, torque is the force that produces or resists rotation. Torque is equal to the product of the force(F) and the distance (r) from the axis of the rotation. Equation:

$$T = F \times r \times \sin(\theta), \quad (1)$$

where T is torque, the lever arm length, θ is the angle between the force and the lever arm.

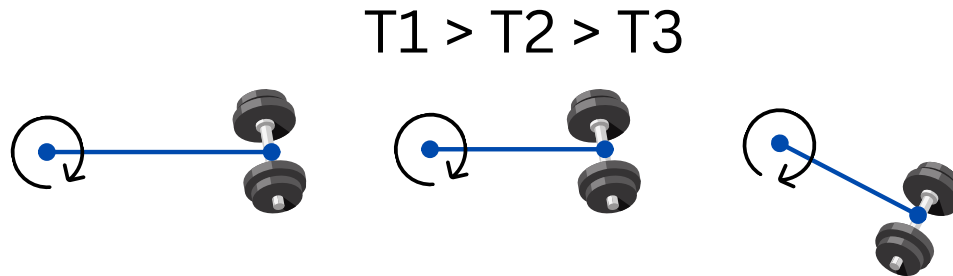


Figure: Torque illustration

Torque-speed curve

The Torque is inverse proportional to the speed.

No-load speed is the maximum speed that the motor can rotate with.

Stall torque is the maximum torque that a motor can produce when the rotor (armature) is prevented from rotating.

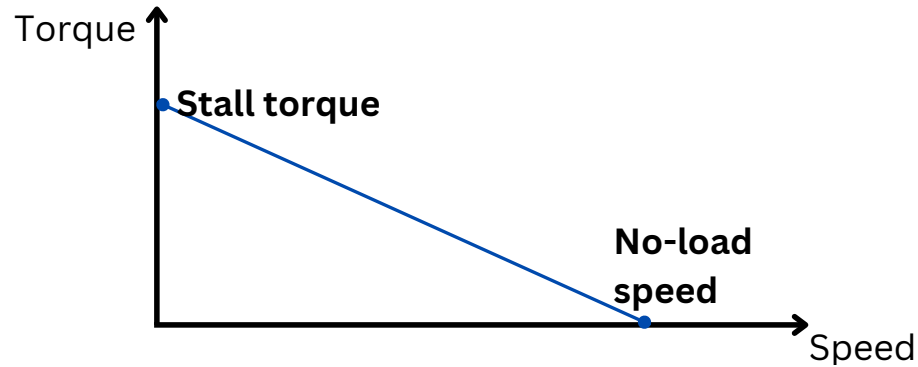


Figure: Torque-speed curve

Why do we need a gearbox?

Using the gearbox allows us to adjust torque-speed ratio to produce more torque. The gearbox ratio dictates

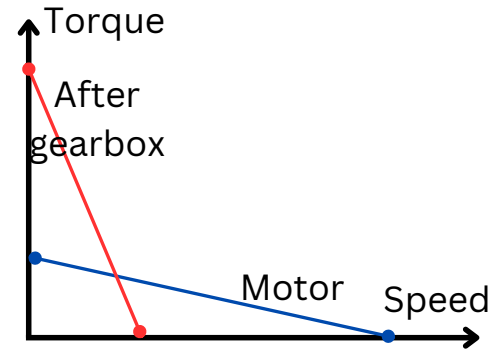


Figure: How gearbox changes torque-speed curve

Torque-current and Speed-voltage relation

Torque is proportional to the applied current:

$$T = K_t i_a \quad (2)$$

Speed is proportional to the armature voltage:

$$e = K_e \dot{\theta}_m \quad (3)$$

Motor Speed:

$$\dot{\theta}_m = \frac{v_a - TR_a/K_t}{K_e} \quad (4)$$

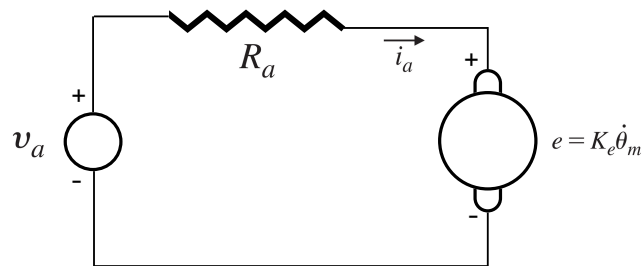


Figure: An electric circuit of an armature

MOSFET?

A transistor is necessary to provide enough current to drive the motor. The microcontroller cannot generate a sufficient current level to drive the motors.

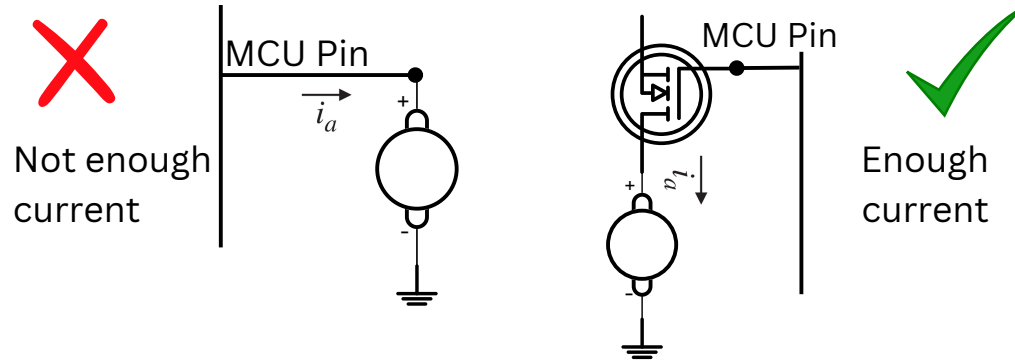


Figure: Direct connection vs MOSFET

H-bridge

H-bridge is necessary to control the direction of the motor rotation

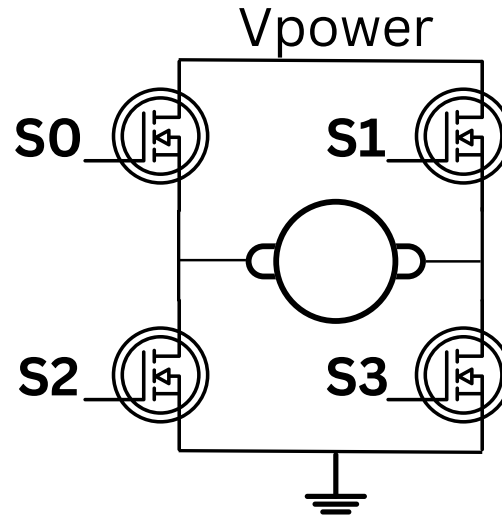


Figure: H-bridge connection

PWM signal

Pulse-width-modulation signal is needed to generate an analog signal using the digital GPIO pin.

Duty cycle

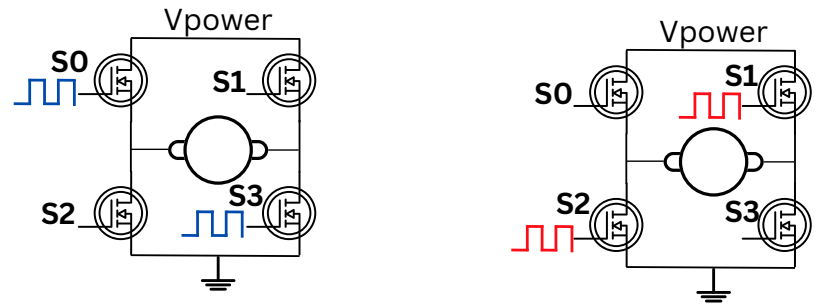
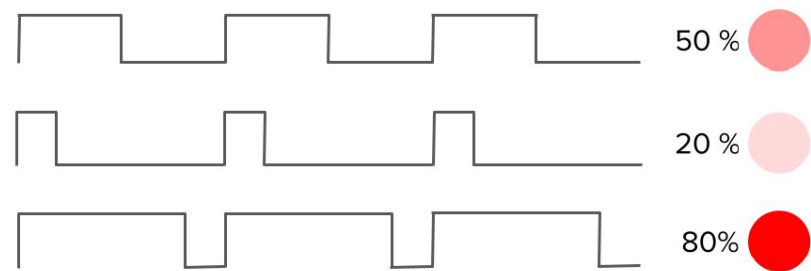


Figure: H-bridge connection

Motor Drivers

Motor Driver is an electronic circuit to provide a sufficient level current to driver a motor. It allows to control the motor using a low-power signal, including PWM. In addition, it ensures over-current and over-temperature protection. Motors Drivers:

1. STSPIN240 - dual channel, 1.3 A, STMicroElectronics
2. STSPIN250 - single channel, 2.5 A,
3. TB6612 - dual channel, 1.2 A, Sparkfun
4. TB6612 - dual channel, 1.2 A, Adafruit
5. MC33926 - single/dual channel, 3A, Pololu
6. L298N - dual channel, 2A, DFRobot

Incremental Encoders

Incremental encoders reports the position change and direction of movement. The encoders are object detectors at a basic level

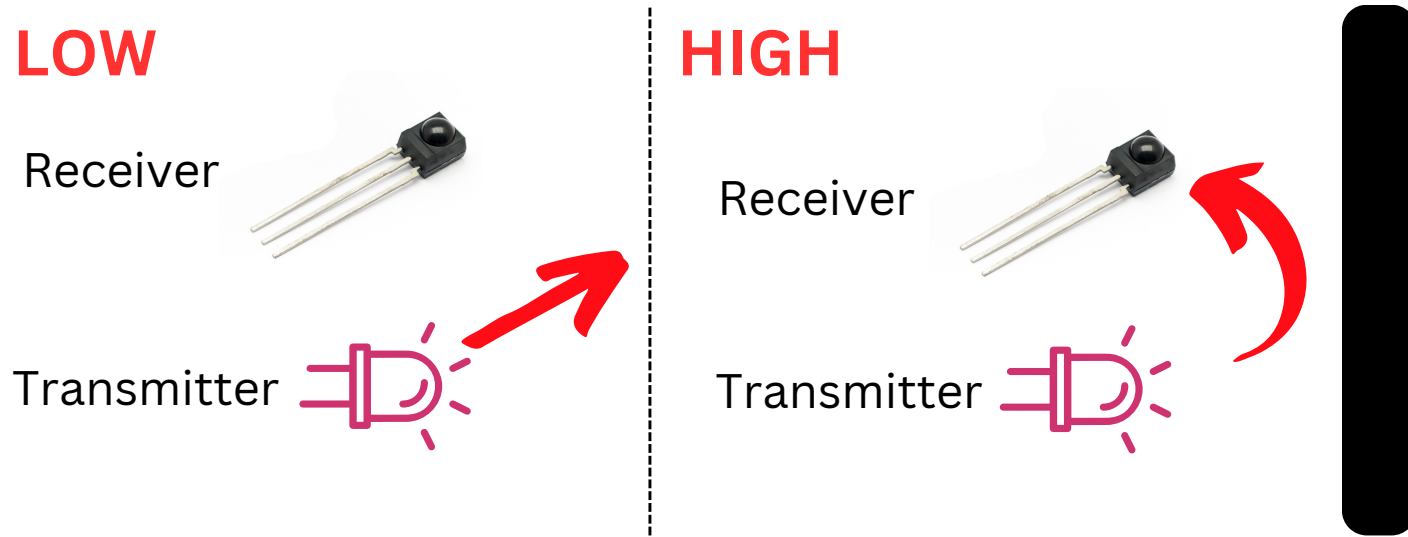


Figure: Proximity Sensor

Incremental Encoders

Encoders sense the presence of lines, and output square-shaped (Quadrature) signals with edges according to the number of lines passed.

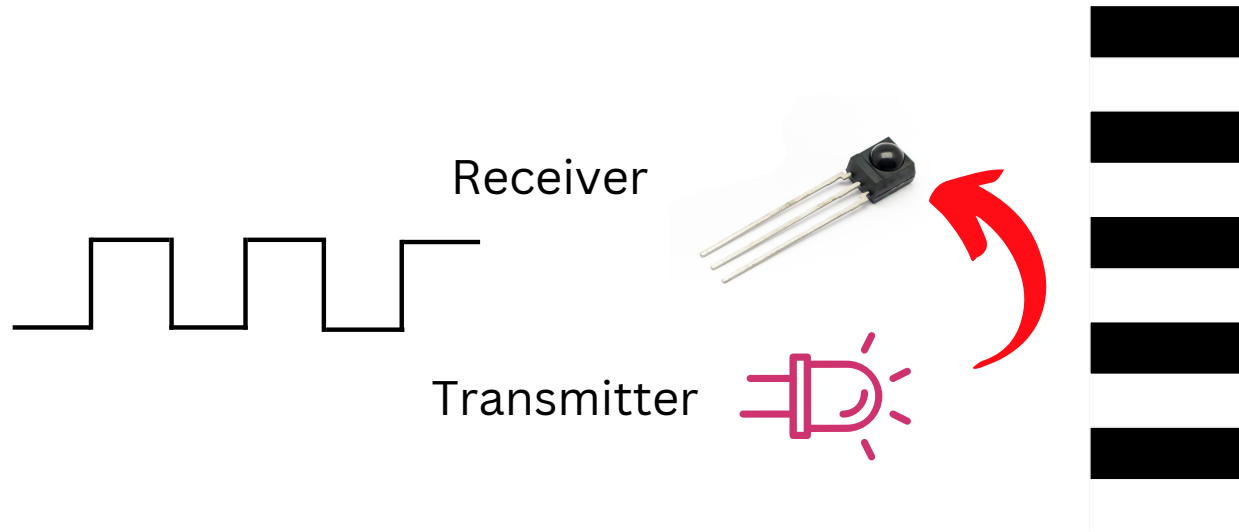
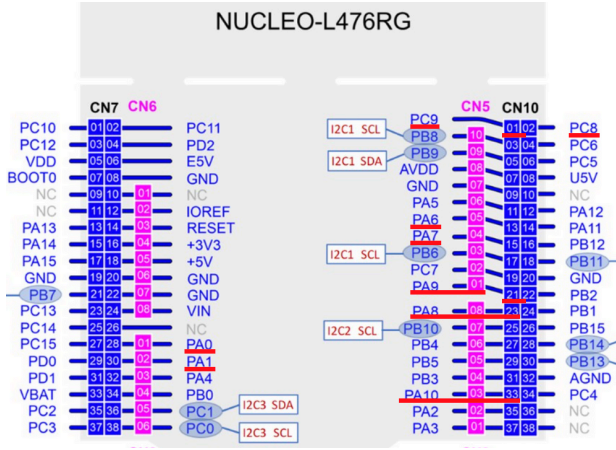


Figure: Linear Encoder Illustration

Motor PWM and Encoder Pinouts

PIN Name	Function
PC9	PHA
PA8	PWMA
PA10	PHB
PA9	PWMB
PC8	RST



PIN Name	Function
PA6	MA ENC1
PA7	MA ENC 2
PA0	MB ENC1
PA1	MB ENC2

Figure: PWM and Encoder Pinout

Moving Average Filter

The purpose of the moving average filter is to denoise the signal.

It is an efficient and simple low-pass filter

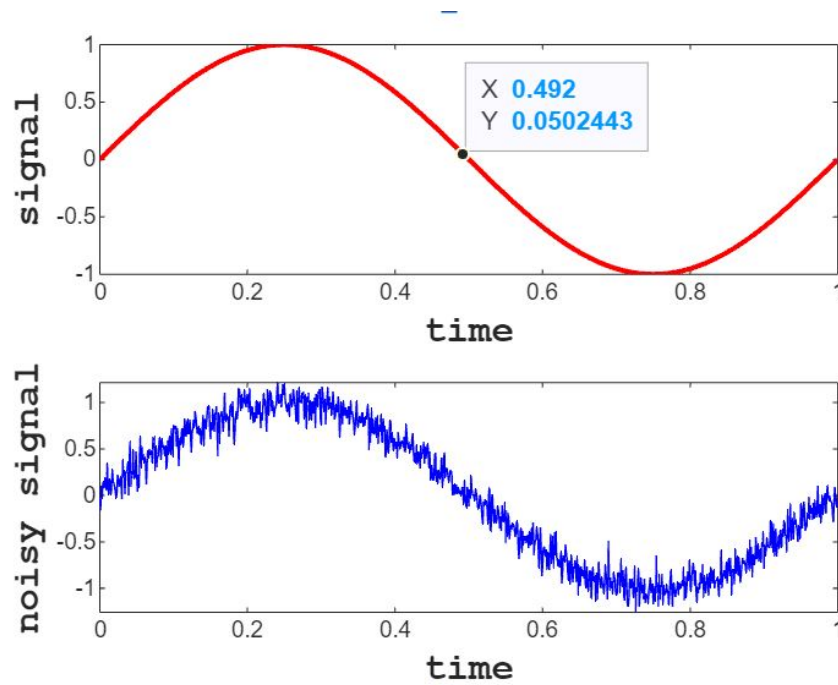


Figure: Clear and noisy signal

Moving Average Filter

A sequence of samples: $a[0], a[1], a[2], a[3], \dots$

Moving average Filter output:

$$b[n] = \frac{a[n] + a[n-1] + \dots + a[n-N+1]}{N}, \quad (5)$$

where N - window length

Moving Average Filter

A sequence of samples: $a[0], a[1], a[2], a[3], \dots$

Moving average Filter output:

$$b[n] = \frac{a[n] + a[n-1] + \dots + a[n-N+1]}{N}, \quad (6)$$

where N - window length

Moving Average Filter

A sequence of samples: $a[0], a[1], a[2], a[3], \dots$

Moving average Filter output:

$$b[n] = \frac{a[n] + a[n-1] + \dots + a[n-N+1]}{N} = \frac{S[n]}{N}, \quad (7)$$

where

$$s[n] = a[n] + a[n-1] + \dots + a[n-N+1] = a[n] + (a[n-1] + \dots + a[n-N+1] + a[n-N]) - a[n-N]$$

From that we obtain a recursive equation:

$$S[n] = a[n] + S[n-1] - a[n-N] \quad (8)$$

Finally:

$$b[n] = \frac{a[n] + S[n-1] - a[n-N]}{N}, \quad (9)$$

Moving Average Filter, pseudo code

```
1
2      #define N    10
3      float window[N];
4      uint8_t counter = 0;
5      float apply_filter(float new_data)
6      {
7          static float sum;
8          sum = sum + new_data - window[counter];
9          window[counter] = new_data;
10         counter++;
11         if(counter == N)
12         {
13             counter = 0;
14         }
15         return sum / N
16     }
```

Integrating the RC Joystick

The receiver of the RC Joystick generates a PWM signal with varying duty cycle. Our task is to identify the duty cycle of the PWM signals to find out the position of the rods.

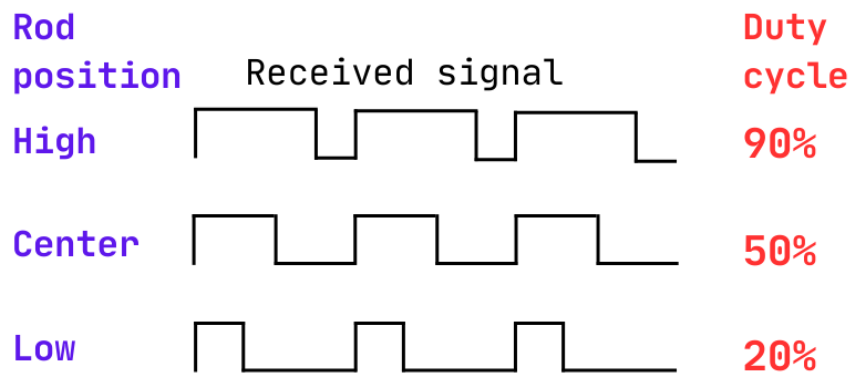
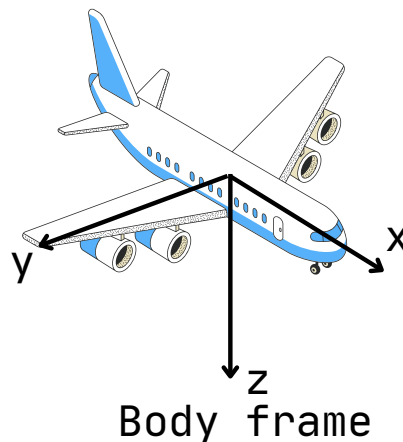
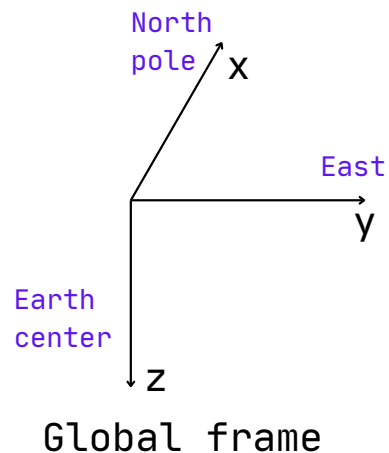


Figure: PWM Signal

Attitude estimation Chapter

1. Serial Peripheral Interface
2. Direct Memory Access
3. Inertial Measurement Unit
4. Normalization and Bias removal
5. A notion of frame and Euler angles



State-Space Design

Advantages of state-space design are especially apparent when the system to be controlled has more than one control input or more than one sensed output.

$$\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}u \quad (10)$$

$$\mathbf{y} = \mathbf{C}\mathbf{x} + \mathbf{D}u \quad (11)$$

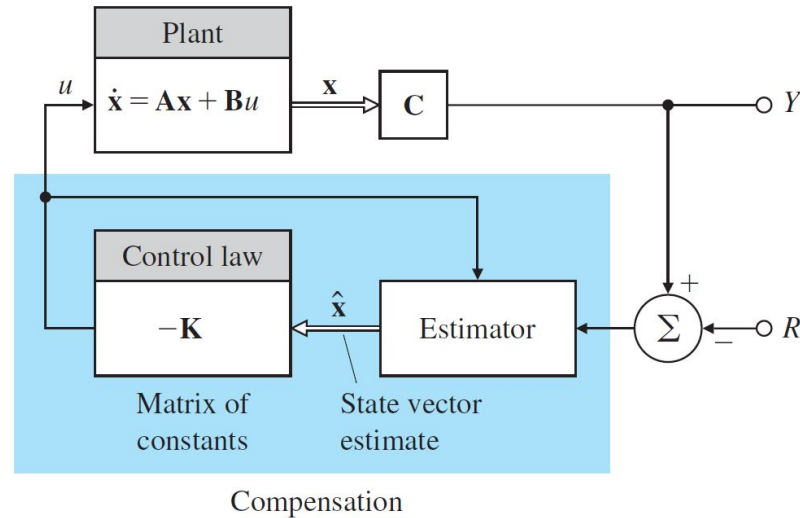


Figure: Compensation Design

State-Space Design

The column vector $\mathbf{x}$ is the state of the system and contains $\mathbf{n}$ elements for an n th-order system. For mechanical systems, the state vector elements usually consist of the positions and velocities of the separate bodies.

A is an $n \times n$ system matrix

B is an $n \times 1$ input matrix

C is a $1 \times n$ output matrix

D is the direct transmission term.

$$\dot{\mathbf{x}} = A\mathbf{x} + Bu$$

$$\mathbf{y} = C\mathbf{x} + Du$$

The Balancing robot state

The state vector of the robot contains a tilt angle, the angular velocity, position, and velocity:

$$\mathbf{x} = \begin{bmatrix} \theta \\ \dot{\theta} \\ x \\ \dot{x} \end{bmatrix} \quad (12)$$

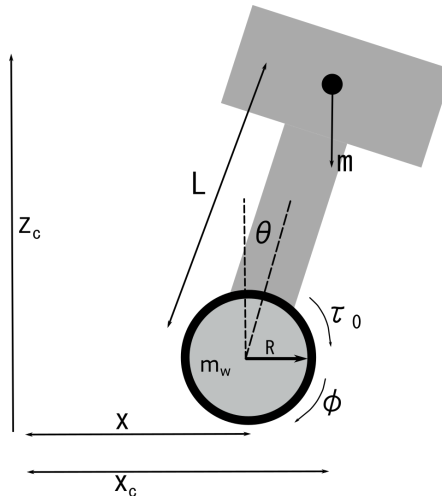


Figure: Robot Illustration

Linear Quadratic Regulator

Linear Quadratic Regulator is widely used method of linear control systems design. The aim of the LQR is to find the control which minimizes the cost function:

$$\mathcal{J} = \int_0^{\infty} [x^T Q x + u^T R u] dt \quad (13)$$

We utilize MATLAB to implement LQR:

```
1 [K,S,P] = lqr(sys,Q,R,N)
2 [K,S,P] = lqr(A,B,Q,R,N)
```
